

Essential AWK Scripting

A Practical Guide to Text Processing and Data Extraction

AWK is a powerful programming language designed for text processing and typically used as a data extraction and reporting tool. This guide covers the most essential one-liners and scripts.

1. Print Specific Columns

AWK automatically splits each line into fields based on whitespace. Use \$1 for the first column, \$2 for the second, and so on. \$0 represents the entire line.

```
# Assuming 'data.txt' contains: "Alice 25 Sales", "Bob 30 IT"
```

```
user@ubuntu:~/data$ awk '{print $1, $3}' data.txt
```

```
Alice Sales
```

```
Bob IT
```

2. Use a Custom Field Separator (-F)

By default, AWK uses spaces or tabs to separate fields. You can change this using the -F flag. This is useful for CSVs or system files like /etc/passwd.

```
user@ubuntu:~$ awk -F":" '{print $1, $3}' /etc/passwd | head -n 3
```

```
root 0
```

```
daemon 1
```

```
bin 2
```

3. Filter Rows by a Condition

You can evaluate conditions before taking an action. Only lines that match the condition will be processed.

```
# Assuming 'scores.txt' contains Name and Score

user@ubuntu:~/data$ awk '$2 >= 80 {print $1, "Passed!"}' scores.txt
Sarah Passed!
Michael Passed!
```

4. Pattern Matching (Regex)

Search for specific strings or regular expressions by placing the pattern between slashes / pattern/. It works similarly to grep.

```
user@ubuntu:/var/log$ awk '/ERROR/ {print $0}' syslog
[10:45:01] ERROR: Connection timed out.
[11:02:15] ERROR: Disk space critically low.
```

5. Line Numbers (NR) and Field Counts (NF)

AWK has built-in variables. NR keeps track of the current Record Number (line number). NF holds the Number of Fields (columns) in the current line.

```
user@ubuntu:~/data$ awk '{print NR, "-", $0, "(Fields: " NF ")"}' data.txt
1 - Alice 25 Sales (Fields: 3)
2 - Bob 30 IT (Fields: 3)
```

6. Perform Math and Accumulate Totals

AWK handles arithmetic easily. You can create variables on the fly. The END block executes only once after all lines have been processed.

```
# Calculate the sum of column 2 (e.g., salaries or sizes)

user@ubuntu:~/data$ awk '{sum += $2} END {print "Total Sum:", sum}'
numbers.txt
Total Sum: 4550
```

7. The BEGIN and END Blocks

BEGIN runs before any input is read (great for headers). END runs after all input is read (great for summaries).

```
user@ubuntu:~/data$ awk 'BEGIN {print "NAME DEPT"} {print $1 " " $3}
END {print "--- DONE ---"}' data.txt
NAME    DEPT
Alice   Sales
Bob      IT
--- DONE ---
```

8. Find Length of the Longest Line

You can use string functions like `length()`. Here we find and print the character length of the longest line in a file.

```
user@ubuntu:~/data$ awk '{ if (length($0) > max) max = length($0) } END
{ print max }' file.txt
124
```

9. Formatted Output with printf

For highly structured tabular output, use `printf` (similar to C). `%-10s` means a left-justified string of 10 characters, and `%5d` means a 5-digit integer.

```
user@ubuntu:~/data$ awk '{printf "%-10s | %5d", $1, $2}' data.txt
Alice      |    25
Bob        |    30
Charlie    |    42
```

10. Find and Replace using gsub()

Use `gsub(target, replacement, target_string)` for global substitution. If you omit the target string, it defaults to the entire line (`$0`).

```
# Replace all occurrences of "apple" with "orange"

user@ubuntu:~/data$ awk '{gsub(/apple/, "orange"); print}' fruits.txt
I have an orange and she has an orange pie.
```